

Vermont Health Platform — Operations & Scaling Guide

Audience: Non-developer business owner **Last updated:** March 2026 **Stack:** Next.js 14 (Vercel) + FastAPI Python (Railway) + Supabase + Sanity + Stripe

Table of Contents

- 1. [Daily Operations Checklist](#)
- 2. [Monitoring](#)
- 3. [Backups](#)
- 4. [Security](#)
- 5. [Scaling — When and How](#)
- 6. [Cost Projection](#)
- 7. [Performance](#)
- 8. [Deploying Updates](#)
- 9. [Incident Response](#)

1. Daily Operations Checklist

Every Day (5 minutes)

1. Check the AI backend is alive

Open your browser and go to:

```
https://your-railway-app.railway.app/api/health
```

You should see a JSON response like `{"status": "ok", "model": "llama-3.3-70b-versatile"}`. If you get an error or a timeout, the Python backend is down. Jump to [AI backend is down](#).

2. Check Vercel deployment status

Go to: <https://vercel.com/dashboard> Look at your project. The status badge next to the latest deployment should say **Ready** (green). If it says **Failed** (red), click it to read the build error log.

3. Scan Vercel error logs

Go to: <https://vercel.com/dashboard> → your project → **Logs** tab Filter by **Error**. Scan for any spike in 4xx or 5xx errors compared to yesterday. A handful of 404s is normal (bots). A sudden surge of 500s is not — investigate.

4. Check Supabase status

Go to: https://supabase.com/dashboard/project/YOUR_PROJECT_REF/reports Check the **API requests** chart. Confirm the database is responding. Also check: <https://status.supabase.com> — if there's an active incident, that explains any anomalies.

Every Week (15 minutes)

1. Log into Supabase to prevent free-tier pausing

Supabase pauses free-tier projects after **7 days of inactivity** (no API calls AND no dashboard visit). Simply visiting https://supabase.com/dashboard/project/YOUR_PROJECT_REF counts as activity. Once you have real users making requests, this becomes automatic. Until then, visit weekly.

2. Review Stripe payments

Go to: <https://dashboard.stripe.com/payments> Check for:

- Any **failed** or **disputed** payments (shown in red)
- Subscription churn (cancellations in the **Subscriptions** tab)

- Any **refund requests** in your email

3. Review Railway resource usage

Go to: <https://railway.app/dashboard> → your service → **Metrics** tab Check that memory usage is below 80% of your plan's limit. If it's consistently above 70%, you are approaching a crash threshold — see [Railway scaling](#).

4. Check Google AI Studio quota

Go to: <https://aistudio.google.com> → **API usage** or <https://console.cloud.google.com/apis/api/generativelanguage.googleapis.com/quotas> Confirm you have not hit daily or per-minute limits. Quota errors show up as 429 responses in Railway logs.

Every Month (30 minutes)

1. Review user growth

Go to: https://supabase.com/dashboard/project/YOUR_PROJECT_REF/auth/users Count total users. Compare to last month. Note which month crossed 500, 1,000, and 10,000 MAU — these are upgrade triggers (see Section 5).

2. Audit API costs

Service	Where to check
Groq (LLM)	https://console.groq.com
Google Gemini (embeddings)	https://console.cloud.google.com/billing
Vercel	https://vercel.com/account/billing
Railway	https://railway.app/account/billing
Supabase	https://supabase.com/dashboard/account/billing
Sanity	https://sanity.io/manage → your project → Billing
Stripe	https://dashboard.stripe.com/balance

Compare each to last month. Any service that jumped more than 20% unexpectedly is worth investigating.

3. Rotate secrets (every 90 days minimum)

See [Rotating API Keys](#) in Section 4 for the full procedure. Mark your calendar now: set a recurring reminder every 90 days.

4. Test backup restoration

Once a month, export a small table from Supabase (see Section 3) and verify the file opens correctly. This confirms backups are working before you ever need them in an emergency.

5. Review Sanity content and assets

Go to: <https://sanity.io/manage> → your project → **Usage** Check asset storage. Free tier allows 10GB. If you're uploading videos or large images, this fills faster than expected.

2. Monitoring

Where to See Errors

Vercel (Frontend errors) URL: <https://vercel.com/dashboard> → your project → **Logs**

- Real-time logs stream here. You can filter by function (e.g., `/api/chat`), status code, or time range.
- Build errors appear under **Deployments** — click any deployment to see its full build log.
- Runtime errors (JavaScript exceptions in server components) appear as 500s in the log stream.

Railway (Python backend errors) URL: <https://railway.app/dashboard> → your service → **Logs**

- All `uvicorn` request logs appear here: method, path, response time, status code.
- Python tracebacks (crashes) appear here in full — search for `Traceback` or `ERROR`.

- If the service restarts unexpectedly, the previous session's final logs appear before the restart line.

Supabase (Database errors) URL: https://supabase.com/dashboard/project/YOUR_PROJECT_REF/logs/postgres-logs

- Raw Postgres query logs (connection errors, slow queries, lock waits).
- For API-level errors (Auth failures, RLS violations), check: **Logs → API logs**

Stripe (Payment errors) URL: <https://dashboard.stripe.com/logs>

- Every API call Stripe receives is logged here, including failures.
- Webhook delivery attempts (successes and failures) are at: <https://dashboard.stripe.com/webhooks>

What the /api/health Endpoint Tells You

The health endpoint lives at:

```
https://your-railway-app.railway.app/api/health
```

A healthy response looks like:

```
{
  "status": "ok",
  "model": "llama-3.3-70b-versatile",
  "index_ready": true,
  "embedding_model": "gemini-embedding-001",
  "vector_store": "pgvector"
}
```

What each field means:

- **status: "ok"** — The FastAPI process is running and responding.
- **model** — The Groq LLM model currently in use. If this changes unexpectedly, a config variable may have been overwritten.
- **index_loaded: true** — The LlamaIndex vector store loaded successfully from **backend/storage/**. If **false**, the AI will not be able to answer questions based on your content — the index needs to be rebuilt (restart the Railway service and watch logs for index-building output).
- **document_count** — Number of documents indexed. If this drops significantly, the RAG content sync may have failed.

If the endpoint returns a **502 Bad Gateway**, the Railway service is down or starting up. Wait 30 seconds and try again. If it is still down after 2 minutes, check Railway logs.

Key Metrics to Watch

Metric	Where to find it	Warning threshold
Monthly Active Users	Supabase → Auth → Users	Approaching 50,000 (free tier limit)
Database size	Supabase → Settings → Database	Approaching 400MB (upgrade before 500MB)
Bandwidth (Vercel)	Vercel → Usage	Approaching 80GB/month
AI chat requests/day	Railway logs (count /api/chat POSTs)	Watch for sudden spikes (abuse)
Groq API errors (429)	Railway logs	Any consistent 429 stream = Groq quota hit
Gemini embedding errors (429)	Railway logs	Only during /api/ingest — embedding quota hit
Stripe MRR	Stripe → Revenue	Track monthly
Subscription churn rate	Stripe → Subscriptions	>5%/month is a problem

Metric	Where to find it	Warning threshold
Build time	Vercel → Deployments	>3 minutes = investigate bundle size

Setting Up Basic Uptime Monitoring (Free)

Option A: UptimeRobot (recommended for beginners)

- Go to <https://uptimerobot.com> and create a free account.
 - Click **Add New Monitor**.
 - Type: **HTTP(s)**
 - Friendly Name: **VT Health Platform – Frontend**
 - URL: **<https://your-vercel-domain.com>**
 - Monitoring Interval: **5 minutes**
 - Click **Create Monitor**.
 - Repeat for the AI backend health endpoint: **<https://your-railway-app.railway.app/api/health>**
 - Under **Alert Contacts**, add your email. You will receive an email within 5 minutes of any outage.

Free plan: 50 monitors, 5-minute intervals, email alerts. This is sufficient for early stage.

Option B: Better Uptime

URL: <https://betteruptime.com> Offers phone call alerts on outages (free plan includes 3 monitors). Useful if you want to be called at 2am rather than just emailed.

3. Backups

Supabase Automatic Backups

Free tier: Point-in-time recovery is NOT available. Supabase takes daily backups on free projects, but restoration requires contacting Supabase support and is not self-service.

Pro tier (\$25/mo): Daily automated backups with self-service restoration from the dashboard. You can restore to any backup from the last 7 days.

To restore from a backup (Pro tier):

- Go to: https://supabase.com/dashboard/project/YOUR_PROJECT_REF/database/backups
 - Select the backup date/time you want to restore to.
 - Click **Restore**. This restores the entire database. It takes 5–30 minutes depending on database size.
 - WARNING: Restoration overwrites your current database. If the issue is a single bad record, use manual export/import instead.

How to Manually Export the Database

Run this command from your terminal (requires **psql** installed, or use the Supabase dashboard):

Via Supabase dashboard (easiest):

- Go to: https://supabase.com/dashboard/project/YOUR_PROJECT_REF/database/backups
 - Click **Download backup** (Pro tier) or use the SQL editor to export specific tables.

Via command line (full dump):

```
pg_dump "postgresql://postgres:
[YOUR_DB_PASSWORD]@db.YOUR_PROJECT_REF.supabase.co:5432/postgres" \
  --no-owner \
  --no-acl \
  -f backup-$(date +%Y-%m-%d).sql
```

Replace **[YOUR_DB_PASSWORD]** and **YOUR_PROJECT_REF** with your actual values from:
https://supabase.com/dashboard/project/YOUR_PROJECT_REF/settings/database

Store the `.sql` file in a safe location (Google Drive, S3, etc.). Automate this monthly at minimum.

Export a single table (e.g., users):

```
psql "postgresql://postgres:
[YOUR_DB_PASSWORD]@db.YOUR_PROJECT_REF.supabase.co:5432/postgres" \
-c "\COPY auth.users TO 'users-export.csv' CSV HEADER"
```

Sanity Content Backup

Sanity does NOT automatically backup your content. You must export it manually.

Export all Sanity content:

```
cd frontend
npx sanity dataset export production sanity-backup-$(date +%Y-%m-%d).tar.gz
```

This creates a compressed archive of all your Sanity documents. Store it alongside your database backup.

Schedule monthly: Put a reminder to run this command monthly and store the output file. The export takes under a minute for typical content volumes.

To restore Sanity content from a backup:

```
npx sanity dataset import sanity-backup-2026-03-19.tar.gz production --replace
```

WARNING: `--replace` overwrites existing documents with the same ID. Use `--missing` to only import documents that don't already exist.

What Is NOT Automatically Backed Up

These items require manual attention:

Item	Risk	What to do
backend/storage/ (LlamaIndex vector index)	If Railway's disk is wiped, AI loses its index and must rebuild from scratch (2–5 min rebuild)	The rebuild is automatic on restart — just monitor logs after deploy
backend/data/ (source PDFs)	If these PDFs are deleted, they cannot be recovered from Railway	Keep originals in a shared drive (Google Drive, Dropbox)
.env files (local dev secrets)	If your laptop dies, you lose local environment configuration	Store in a password manager (1Password, Bitwarden) — NEVER in git
Stripe product/price configuration	Stripe products are not exported by default	Screenshot your product catalog; document price IDs in a secure note
Vercel environment variables	Not auto-backed-up	Export via: Vercel dashboard → Settings → Environment Variables → copy all values to a password manager
Sanity schema definitions	These live in your git repo, so they ARE backed up if you push regularly	Ensure you push schema changes before deploying

4. Security

Rotating API Keys (When and How)

Rotate all secrets every **90 days**, or immediately if you suspect a leak.

Rotation schedule and procedure:

Secret	Where to generate new key	Where to update it
GOOGLE_API_KEY	https://console.cloud.google.com/apis/credentials → Create Credentials → API Key	Railway: service → Variables; local: backend/.env
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase → Settings → API → Rotate anon key	Vercel: Settings → Environment Variables; local: frontend/.env.local
SUPABASE_SERVICE_ROLE_KEY	Supabase → Settings → API → Rotate service_role key	Vercel and local env
STRIPE_SECRET_KEY	Stripe → Developers → API Keys → Roll key	Vercel and local env
STRIPE_WEBHOOK_SECRET	Stripe → Webhooks → your endpoint → Reveal signing secret	Vercel and local env
SANITY_API_TOKEN	https://sanity.io/manage → your project → API → Tokens	Vercel and Railway if used by backend

Procedure for rotating any key:

1. Generate the new key in the service's dashboard.
2. Update the key in Vercel: Project → Settings → Environment Variables → find the variable → Edit → paste new value → Save.
3. Update the key in Railway: Service → Variables → find the variable → edit → save. Railway will restart the service automatically.
4. Update your local .env / .env.local file.
5. Trigger a new Vercel deployment (git push, or Vercel dashboard → Redeploy) so the new env var takes effect.
6. Revoke the old key in the service dashboard immediately after confirming the new one works.
7. Test: visit your site and run a test AI chat query to confirm nothing broke.

What to Do If a Key Is Leaked

A key is "leaked" if it appears in a git commit, a Slack message, a public forum post, or anywhere outside your password manager or deployment environment.

Act within 15 minutes:

1. **Revoke the key immediately** in the originating service's dashboard. Do not wait.
2. **Generate a new key** and deploy it using the rotation procedure above.
3. **Audit recent usage:** most services (Google Cloud Console, Stripe, Supabase) show API call logs — look for any unexpected requests in the last 24–72 hours.
4. **If a SUPABASE_SERVICE_ROLE_KEY was leaked:** This key bypasses Row Level Security and has full database access. In addition to rotating it, check the Supabase Auth logs for unexpected user creation or data access.
5. **If a STRIPE_SECRET_KEY was leaked:** Check Stripe logs immediately for any charges or payout changes you did not initiate. Contact Stripe support if you see anything suspicious.
6. **Remove the key from git history** (if it was committed):

```
git filter-branch --force --index-filter \  
  'git rm --cached --ignore-unmatch .env' \  
  --prune-empty --tag-name-filter cat -- --all  
git push origin --force --all
```

This is a destructive git operation — make sure collaborators know before running it.

Row Level Security (RLS) is a database feature that controls which rows a user can read or write, based on who they are authenticated as.

What RLS protects:

- Users can only read and modify **their own** profile data, saved content, and subscription status.
- Even if someone discovers the Supabase URL and anon key (both are public in the frontend), they cannot read another user's data.
- Without RLS enabled on a table, any authenticated user could query any row. With RLS, policies restrict access per-user.

What RLS does NOT protect:

- The `SUPABASE_SERVICE_ROLE_KEY` bypasses RLS entirely. Guard this key like a password — it has full database access. It should only ever be used server-side (in Railway or Vercel server functions), never exposed to the browser.
- Tables without RLS policies enabled are open to all authenticated users. Run this query in the Supabase SQL editor to check:

```
SELECT tablename, rowsecurity
FROM pg_tables
WHERE schemaname = 'public';
```

Any row with `rowsecurity = false` on a table containing user data is a security gap.

Rate Limiting

What is currently in place:

Layer	Protection	Limit
Vercel Edge Network	DDoS protection, bot filtering	Automatic, managed by Vercel
Next.js API routes	Per-IP rate limiting (if implemented)	Depends on your <code>middleware.ts</code> implementation
Railway / FastAPI	No rate limiting by default	You must add it (see below)
Groq API	Per-minute and per-day quotas	~6,000 tokens/min free tier; upgrade at console.groq.com
Supabase Auth	Login attempt throttling	Built-in: 30 failed logins triggers a lockout

Adding rate limiting to the Python backend (recommended):

Install `slowapi` in your backend:

```
pip install slowapi
```

Add to `backend/main.py`:

```
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

@app.post("/api/chat")
@limiter.limit("20/minute") # 20 requests per minute per IP
```



```
async def chat(request: Request, ...):  
    ...
```

This prevents a single user from sending hundreds of AI queries per minute (which would drain your Groq quota).

CSP Headers — What They Protect

Content Security Policy (CSP) headers tell browsers which sources of scripts, styles, and data are trusted. They protect against **Cross-Site Scripting (XSS)** attacks — where malicious code injected into your page tries to steal user data or session tokens.

What CSP headers protect:

- Prevents injected `<script>` tags from loading code from attacker-controlled domains.
- Prevents your page from being embedded in an iframe on another site (clickjacking).
- Blocks inline scripts unless explicitly allowed.

Where to configure them: `frontend/next.config.ts` → `headers()` function.

Current recommended CSP for this stack:

```
{  
  key: 'Content-Security-Policy',  
  value: [  
    "default-src 'self'",  
    "script-src 'self' 'unsafe-inline' https://js.stripe.com",  
    "style-src 'self' 'unsafe-inline'",  
    "img-src 'self' data: https://cdn.sanity.io https://*.supabase.co",  
    "connect-src 'self' https://*.supabase.co https://api.stripe.com https://your-  
railway-app.railway.app",  
    "frame-src https://js.stripe.com",  
  ].join('; ')  
}
```

Note: `'unsafe-inline'` weakens CSP for scripts but is often required for Next.js inline scripts and some third-party tools. The highest priority is ensuring `connect-src` only allows your known backend domains.

5. Scaling — When and How

Supabase

Free tier limits:

- Database storage: **500MB**
- Monthly Active Users: **50,000 MAU**
- Bandwidth: **2GB/month**
- No daily backups (manual only)
- Project pauses after 7 days of inactivity

When to upgrade to Pro (\$25/month):

Upgrade when ANY of the following occurs:

- Database size reaches **400MB** (give yourself a buffer before the 500MB hard limit)
- MAU reaches **40,000** (upgrade before the 50,000 cutoff, not after)
- Monthly bandwidth approaches **1.5GB**
- You need reliable daily backups (self-service restoration)
- You cannot risk a paused project (i.e., you have paying users)

Pro tier features that matter:

- No project pausing — ever
- Daily automated backups with 7-day retention, self-service restoration

- 8GB database storage (vs. 500MB)
- 250GB bandwidth (vs. 2GB)
- Priority email support

At expected growth rates, you will likely hit the MAU limit before storage or bandwidth. Track MAU monthly.

Vercel

Free tier (Hobby plan) limits:

- Bandwidth: **100GB/month**
- Serverless function executions: **100,000/month**
- Build time: **6,000 minutes/month**
- Custom domains: unlimited
- Team members: 1

When to upgrade to Pro (\$20/month per member):

- Bandwidth exceeds **80GB/month** consistently
- You need **password protection** for preview deployments (useful for sharing staging with clients)
- You need team access for multiple developers
- Build failures due to timeout (Pro allows longer build times)
- You want **Web Analytics** (included in Pro)

Note: 100GB/month of bandwidth supports approximately 100,000–500,000 page views per month (depending on page size). This is generous for early stage. Most teams don't need Vercel Pro until 6–18 months in.

Railway (Python Backend)

Free tier:

- \$5 credit per month (roughly 500 service-hours at minimal resource usage)
- Shared CPU, 512MB RAM by default

How to size the service:

The Python backend (FastAPI + LlamaIndex) uses:

- **CPU:** Mostly idle between requests; spikes during index rebuilds
- **RAM:** ~200–400MB baseline when the index is loaded; spikes to ~600MB during rebuild

Recommended starting configuration:

- Plan: **Hobby (\$5/mo)** — gives you \$5 monthly credit, which covers ~720 hours of a small service
- RAM: 512MB (increase to 1GB if you see OOM restarts in logs)
- CPU: 1 vCPU shared (sufficient for low traffic)

When to scale up:

- Memory consistently above 400MB (approaching crash territory) → increase to 1GB RAM
- Response times above 5 seconds consistently → increase CPU allocation
- More than ~100 concurrent users → consider moving to a dedicated plan

To resize on Railway: Go to: Railway dashboard → your service → **Settings** → **Resource Limits** → adjust RAM/CPU sliders.

Groq (Chat LLM)

Free tier limits (as of 2026):

- **llama-3.3-70b-versatile**: 6,000 tokens/minute, 1,000+ requests/day
- **llama-3.1-8b-instant**: higher throughput, lower quality — fallback option

How to upgrade: Go to <https://console.groq.com> → Billing → upgrade to a paid plan. Paid tier limits are substantially higher and priced per million tokens.

Cost estimate per 1,000 AI chat queries (approximate, 2026 pricing):

Assumptions: Average query = 500 input tokens + 300 output tokens = 800 tokens per query. 1,000 queries = 800,000 tokens.

Model	Input cost	Output cost	Est. cost per 1,000 queries
llama-3.3-70b-versatile	~\$0.59/million tokens	~\$0.79/million tokens	~\$0.11
llama-3.1-8b-instant	~\$0.05/million tokens	~\$0.08/million tokens	~\$0.01

For early stage (under 1,000 total chat queries per day), the free tier is sufficient. At 10,000 queries/day, monthly cost on paid Groq ≈ **\$33/month** (Llama 3.3 70B).

Google Gemini (Embeddings Only)

Free tier limits (as of 2026):

- gemini-embedding-001: 1,500 requests/day, 5 requests/second

Embeddings are only called during index builds (via `/api/ingest`), not during chat. The free tier is sufficient for normal re-indexing cadence.

How to upgrade: Enable billing on your Google Cloud project at <https://console.cloud.google.com/billing> — usage-based pricing applies beyond free tier.

Sanity

Free (Developer) tier limits:

- Admin users: **3**
- Non-admin users: **unlimited** (read-only API access)
- Asset storage: **10GB**
- API requests: **250,000/month**
- Bandwidth: **10GB/month**

When to upgrade to Team (\$99/month):

- You need more than 3 editors in Sanity Studio
- Asset storage approaches **8GB** (upgrade before hitting the limit)
- You need role-based access control (assigning different permissions to different editors)
- API requests approach **200,000/month** (high-traffic public site querying Sanity directly)

Note: The Team plan at \$99/month includes 25GB assets, unlimited admin users, and advanced roles. For a content team of 1–3 people, the free tier typically lasts 12–24 months.

6. Cost Projection

All estimates assume typical usage patterns. "Free" means within free tier limits.

0–100 Users: All Free Tiers (~\$0/month)

Service	Cost	Notes
Vercel	\$0	Well within free limits
Railway	\$0	\$5 free credit covers a small always-on service
Supabase	\$0	Database tiny, MAU minimal
Sanity	\$0	Well within free tier
Groq (LLM)	\$0	Free tier: 1,000+ req/day
Google Gemini (embeddings)	\$0	Free tier covers re-indexing needs
Stripe	\$0 + 2.9% + \$0.30 per transaction	Only pay when you earn

Service	Cost	Notes
Total	~\$0/month	

This stage: focus on building content, onboarding first users, validating product-market fit.

100–1,000 Users: First Paid Tier (~\$50–100/month)

At 1,000 MAU, Supabase free tier is fine (50,000 MAU limit). But you likely have:

- Paying subscribers generating Stripe revenue
- A team member or two needing Sanity access
- Enough AI usage to consider paid Groq tier for higher rate limits

Service	Cost	Trigger
Vercel	\$0–\$20	Upgrade if bandwidth exceeds 80GB or you need a team
Railway	\$5–\$20	Upgrade if memory usage spikes; Hobby plan at \$5
Supabase	\$0	Still within free tier
Sanity	\$0	Still within free tier (if ≤3 editors)
Groq (LLM)	\$0–\$10	Pay-as-you-go if you hit free tier limits; minimal cost at this scale
Google Gemini (embeddings)	\$0	Free tier sufficient for re-indexing
Stripe	2.9% + \$0.30/transaction	Revenue-dependent
Total	~\$5–50/month	

This stage: prove revenue covers infrastructure costs. Stripe revenue should comfortably exceed hosting costs.

1,000–10,000 Users: Full Production Stack (~\$200–400/month)

At this scale, you need reliability guarantees: no paused databases, daily backups, and faster AI.

Service	Cost	Why
Vercel Pro	\$20/month	Team access, higher limits, Web Analytics
Railway	\$20–50/month	Dedicated resources, higher RAM for AI backend
Supabase Pro	\$25/month	No pausing, daily backups, 8GB storage
Sanity Team	\$0–\$99/month	Only if you have >3 editors or >10GB assets
Groq (LLM)	\$10–50/month	~10,000–100,000 queries/month at Llama 3.3 70B rates
Google Gemini (embeddings)	\$0–5/month	Pay-as-you-go if frequent large re-indexes
Stripe	2.9% + \$0.30/transaction	Revenue-dependent
Total	~\$115–400/month	

At 5,000 paying subscribers at \$30/month, revenue = \$150,000/month. Infrastructure at \$400/month = 0.3% of revenue. This is a healthy ratio.

At this stage, also add:

- Sentry.io for error tracking (~\$26/month): <https://sentry.io> — catches JavaScript and Python errors before users report them
- SendGrid or Postmark for transactional email (~\$15/month): more reliable than Supabase built-in email at scale

7. Performance

What Makes Pages Slow

- 1. Unoptimized images** Images without explicit `width` and `height` on Next.js `<Image>` components cause layout shift and slow loads. Fix: Always use Next.js `<Image>` with width/height. Sanity CDN auto-serves WebP if you use the Sanity image URL builder.
- 2. Large JavaScript bundles** Too many npm packages imported in client components bloat the bundle. Fix: Use `dynamic(() => import('./HeavyComponent'), { ssr: false })` for large components only needed after page load (e.g., interactive maps, charts).
- 3. Slow Sanity queries** Fetching too many fields or deeply nested references in GROQ queries slows response time. Fix: Project only the fields you need. Instead of `*[_type == "post"]`, use `*[_type == "post"]{ title, slug, publishedAt }`.
- 4. No caching on data fetches** Every page load hitting Sanity's API adds 200–500ms per request. Fix: Use Next.js `revalidate` (see below).
- 5. Cold starts on Railway** If Railway scales your service to zero between requests, the first request after idle takes 5–10 seconds. Fix: On paid Railway plans, set minimum instances to 1 to prevent scale-to-zero.

Next.js Caching Strategy (revalidate Settings)

In `frontend/app/[page]/page.tsx` server components, add a revalidation time:

```
// Revalidate this page at most once every 60 seconds
export const revalidate = 60;
```

Recommended settings by page type:

Page type	revalidate	Rationale
Home page	60 (1 minute)	Ticker, daily insights update frequently
Article pages (<code>/policy/[slug]</code>)	3600 (1 hour)	Content doesn't change often
Academy modules	3600 (1 hour)	Stable educational content
Dashboard / live data	300 (5 minutes)	Balance freshness vs. API calls
Static pages (<code>/mission</code> , <code>/values</code>)	86400 (1 day)	Rarely changes
AI chat (<code>/chat</code>)	Not applicable	Dynamic, no caching

On-demand revalidation (when you publish new content in Sanity): Set up a Sanity webhook that calls your Next.js revalidation endpoint when content is published. Sanity webhook URL to configure: `https://your-vercel-domain.com/api/revalidate?secret=YOUR_SECRET`

When to Add a CDN

Vercel already includes a CDN for static assets (images, fonts, JS bundles). You don't need a separate CDN for most cases.

When you need an additional CDN (e.g., Cloudflare):

- You're serving large video files (Sanity CDN handles images but not video well)
- You have international users with latency complaints (Cloudflare has 200+ global PoPs)
- You want DDoS protection beyond what Vercel provides

To add Cloudflare (free tier):

- Sign up at `https://cloudflare.com`
- Add your domain and update your nameservers to Cloudflare's.
- Enable **Proxy** (orange cloud) on your A/CNAME records.
- Set SSL mode to **Full (strict)**.

This adds global edge caching, DDoS protection, and firewall rules at no cost.

Database Query Optimization Basics

When to add indexes: If a database query takes more than 100ms (visible in Supabase logs), it likely needs an index.

Most common indexes to add:

```
-- If you query posts by published date:
CREATE INDEX idx_posts_published_at ON posts(published_at DESC);

-- If you query by user ID (nearly always needed):
CREATE INDEX idx_subscriptions_user_id ON subscriptions(user_id);

-- If you filter by status:
CREATE INDEX idx_subscriptions_status ON subscriptions(status);
```

Run these in the Supabase SQL editor: https://supabase.com/dashboard/project/YOUR_PROJECT_REF/sql

Check for slow queries: Supabase → Logs → Postgres Logs → filter by `duration > 100ms`. Any query appearing repeatedly in slow logs is a candidate for an index.

Avoid N+1 queries: An N+1 query is when you fetch a list of 100 items, then make a separate database call for each item to get related data. In Sanity, solve this with GROQ projections. In Supabase, solve with SQL JOINS or a single query with `select('*', related_table('*'))`.

8. Deploying Updates

How to Deploy Frontend Changes

The frontend deploys automatically via Vercel's git integration.

Normal workflow:

```
# Make your changes locally
git add frontend/components/HomeSidebar.tsx
git commit -m "update sidebar navigation"
git push origin main
```

That's it. Vercel detects the push to `main`, triggers a build automatically, and deploys within 1–3 minutes. You can watch the build progress at: <https://vercel.com/dashboard> → your project → **Deployments**

Preview deployments: Any push to a non-main branch creates a preview URL (e.g., `your-app-abc123.vercel.app`). Use these to test changes before merging to main.

How to Deploy Backend Changes

The Python backend on Railway also deploys automatically from git.

Normal workflow:

```
# Make changes to backend/ files
git add backend/main.py
git commit -m "add rate limiting to chat endpoint"
git push origin main
```

Railway detects the push, rebuilds the Docker image (or uses buildpacks), and restarts the service. Watch at: <https://railway.app/dashboard> → your service → **Deployments**

Important: After a Railway deploy, the Python backend restarts. If `backend/storage/` (the LlamaIndex index) is persisted to a Railway volume, the service loads the cached index (~2 seconds). If the volume was wiped (e.g., you redeployed from scratch), it rebuilds the index (~2–5 minutes). During rebuild, the `/api/health` endpoint may return `"index_loaded": false`. Wait and refresh.

How to Run Database Migrations on Production

A "migration" is a SQL change to your database schema (adding a column, creating a table, etc.).

Never run migrations by editing the database directly without a backup.

Safe migration workflow:

1. **Test locally first** using the Supabase CLI:

```
supabase db diff --schema public
```

2. **Create a migration file:**

```
supabase migration new add_subscription_tier_column
```

This creates `supabase/migrations/TIMESTAMP_add_subscription_tier_column.sql`. Write your SQL inside it.

3. **Review the SQL carefully.** Destructive operations (`DROP COLUMN`, `DROP TABLE`) cannot be undone without a backup restore.
4. **Apply to production:**

```
supabase db push
```

This runs all pending migrations against your production Supabase database.

5. **Verify** in the Supabase Table Editor that the schema change is correct.

If you don't use the CLI, you can paste SQL directly in the Supabase SQL editor. Take a manual backup first.

How to Roll Back a Bad Deploy

Frontend (Vercel):

1. Go to: `https://vercel.com/dashboard` → your project → **Deployments**
2. Find the last known-good deployment (before the bad change).
3. Click the three-dot menu on that deployment → **Promote to Production**.
4. This instantly re-routes production traffic to the old deployment. No code changes needed.

This is the fastest rollback mechanism available — takes about 10 seconds.

Backend (Railway):

1. Go to: `https://railway.app/dashboard` → your service → **Deployments**
2. Find the last known-good deployment.
3. Click **Rollback** (or **Redeploy**) on that deployment.
4. Railway restarts the service with the old code.

Database (Supabase — most serious):

If a bad migration corrupted data:

1. Immediately take a fresh export: `pg_dump ... -f emergency-backup.sql`
2. Assess: can you write a SQL `UPDATE` or `INSERT` to fix the data without a full restore?

3. If yes: fix in the Supabase SQL editor, document what happened.

4. If no: restore from the most recent backup (Pro tier: self-service; Free tier: contact Supabase support).

Prevention: Always deploy database migrations separately from code changes. Deploy the migration, verify it works, then deploy the code that uses the new schema.

9. Incident Response

Site Is Down — Step-by-Step Diagnosis

If users report the site is unreachable or you get a UptimeRobot alert:

Step 1 — Check if it's a known outage (2 minutes)

- <https://vercel-status.com>
 - <https://status.supabase.com>
 - <https://status.railway.app>
 - <https://status.groq.com> (for Groq LLM)
 - <https://status.google.com> (for Gemini embeddings)

If any of these show an incident, you are caught in a third-party outage. Nothing to do except wait and post a status update to users.

Step 2 — Check Vercel (2 minutes) Go to <https://vercel.com/dashboard> → your project → **Deployments**. Is the latest deployment green (Ready)? If red (Failed): read the build log, find the error, fix it, push again.

Step 3 — Check DNS (1 minute) Run: `ping your-domain.com` If the IP doesn't respond, the domain or DNS may be misconfigured. Log into your domain registrar and verify nameservers/DNS records are unchanged.

Step 4 — Check if the API is the problem (2 minutes) Go to your site's public URL. Does the homepage load (even if slowly)? If yes, it's not a full outage — a specific feature may be broken.

- If the AI chat is broken: check Railway (Step 5).
 - If auth/login is broken: check Supabase.
 - If payments are broken: check Stripe.

Step 5 — Check Railway (3 minutes) Go to <https://railway.app/dashboard> → your service.

- Is the service running (green dot)?
 - Check **Logs** for crash messages (`Traceback`, `OOMKilled`, `SIGKILL`).
 - If crashed: click **Restart** and watch logs.
 - If OOMKilled (out of memory): increase RAM in Settings, then restart.

Time to resolution target: Most outages are resolved within 15–30 minutes by following these steps. If you cannot identify the cause within 30 minutes, post a status update to users and escalate to your developer.

Database Is Down

Signs: Login fails, user data doesn't load, Supabase errors in Vercel logs.

1. Check <https://status.supabase.com> — is it a Supabase platform incident?

2. If yes: wait. Post a status update. No action needed on your end.

3. If no: go to https://supabase.com/dashboard/project/YOUR_PROJECT_REF
 - Is the project **Paused**? (Free tier pauses after 7 days inactivity.) Click **Restore** — takes ~2 minutes.
 - Is the project Active? Check the **Reports** tab — is the DB responding to queries?

4. If the DB is active but queries are failing: check for a migration that may have broken a table or view. Go to **SQL Editor** and run: `SELECT 1;` — if this returns a result, the DB is up and the problem is in application code or schema.

5. Contact Supabase support if the project is active but unresponsive: <https://supabase.com/dashboard/support>

Business continuity during a database outage: The frontend will still render cached/static pages. Only dynamic features (auth, user data, AI with user history) will fail. Consider adding a maintenance banner: a simple environment variable flag that shows a banner when set.

AI Backend Is Down

Signs: `/chat` page errors, health endpoint unreachable, Railway shows service as crashed.

1. Check `https://status.railway.app` — platform outage?
2. Check Railway logs for the crash reason:
 - `OOMKilled` → out of memory. Increase RAM to 1GB in Settings, restart.
 - `ModuleNotFoundError` → a Python dependency is missing. Check `requirements.txt`, push a fix.
 - `GROQ_API_KEY invalid` → the Groq API key expired or was rotated without updating Railway env vars. Update in Railway → Settings → Variables, then redeploy.
 - `GOOGLE_API_KEY invalid` → the Gemini embedding key expired. Update in Railway → Settings → Variables, then redeploy.
 - `Connection refused` from LlamaIndex → likely a transient Groq API issue. Restart the service and check `https://status.groq.com`.
3. Restart the service: Railway dashboard → your service → **Restart**.
4. Watch logs. The service should print `Application startup complete` and then `INFO: Uvicorn running on http://0.0.0.0:8000` within 30 seconds.
5. Verify with the health endpoint: `https://your-railway-app.railway.app/api/health`

While the AI backend is down: The rest of the site functions normally. Only the `/chat` page is affected. Optional: add a fallback message in `frontend/app/chat/page.tsx` that displays when the backend is unreachable, instead of an error.

Stripe Is Down (Payments Failing)

Signs: Subscription sign-ups fail, users cannot update payment methods, webhook errors.

1. Check `https://status.stripe.com` — is it a Stripe incident?
2. Check `https://dashboard.stripe.com/logs` — look for failed API calls and their error codes.
3. Common error codes:
 - `card_declined` — the user's card was declined. Not a Stripe outage — the user should try another card.
 - `api_connection_error` — your server cannot reach Stripe. Check Railway logs for network issues.
 - `invalid_api_key` — your Stripe secret key may have been rotated. Verify the key in Vercel env vars.
4. Check Stripe webhooks: `https://dashboard.stripe.com/webhooks` → your endpoint → **Recent deliveries**. Failed webhook deliveries mean Stripe cannot reach your Next.js API endpoint. Check Vercel logs for the `/api/stripe/webhook` endpoint.

If Stripe has a platform-wide outage: Payments are queued — Stripe retries failed charges automatically. No action needed beyond posting a status update. Stripe's SLA is 99.99% uptime historically; outages are rare and brief.

What to Communicate to Users

During an outage, communicate within 15 minutes. Silence is worse than a brief honest message.

Channels to use:

- Site status page (create one at `https://betteruptime.com` or `https://statuspage.io` — free tiers available)
- Email to affected users (if you have their contact info and the issue affects their subscription)

Template for minor outage (e.g., AI chat down):

 minutes. All other features are working normally. Thank you for your patience."

Template for major outage (site down):

"The Vermont Health Platform is currently experiencing an outage. We are working to restore service as quickly as possible. We apologize for the inconvenience and will provide an update in [30 minutes]."

Template for resolved:

"The [issue] has been resolved as of [time]. All features are now working normally. We apologize for the disruption and have taken steps to prevent this from happening again."

What NOT to say:

- Do not blame third-party vendors publicly (even if it is Supabase's fault).
 - Do not share technical details (infrastructure details can be a security risk).
 - Do not promise a specific fix time you are not confident about — give a "next update" time instead.
-
- End of Operations Guide — Vermont Health Platform For technical questions beyond this guide, contact your developer or open an issue in the project repository.*